

Price Feed

For Receiving Price Feed Data the client application needs to connect to Web Feed Handler

The method of communicating with Web Feed Handler is via industry standard TCP/IP or Web Sockets

The Web Feed Handler IP and Port are available once you login to the application.

The following fields in the Logon Response are to be used

OdinBcastIP as Web Feed Handler IP

OdinBcastPort as Web Feed Handler Port

Each connection between a Third Party API user i.e. client and Web Feed Handler is defined as a session. An API Interface session is made up of three parts:

- session establishment
- broadcast Subscription
- session termination

Network fragmentation need to be handled at Front End Level

(Sample Code at the end of document)

Session Establishment

Session establishment is via a 101 Logon request from a Third-Party API user. Successful establishment of the session will be indicated when the API user receives a 102 successful logon response message on TCP channel. All Subscribe Feed data will be sent from Web Feed Handler to the user's systems only once a valid session has been established.

Session Termination

Session termination will occur when an API user disconnect TCP/IP Socket.

Message Compression

Each message between a third-party client and Broadcast servers has to be compressed using industry standard Zlib compression. Header Packet of Five character needs to prefix with all Compress Message.

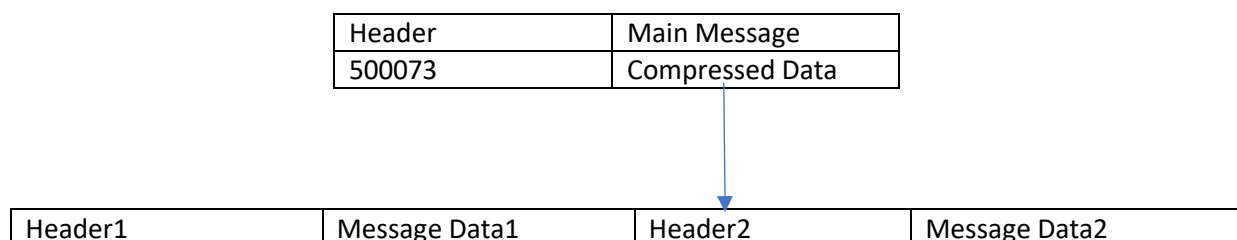
Message Decoding Logic

On Receiving the message, one needs to check the first byte.

The value of 5 in first byte indicates compresses message and value of 2 indicates uncompressed message

The next 5 bytes indicate the length of the message

The compressed Main Message on decompression using Zlib will have multiple Sub Messages each having header and uncompressed message data



Summary of Messages and Message Codes

Message Type	Message Code
Login Request	101
Login Response	102
Best Five Request	127
Best Five Information	128
Touchline Request	206
Touchline Information	209

Message Structures

All Messages are composed of the standard header followed by the body. Native API has been modelled on the lines of FIX i.e. the convention followed is tag=value pair for any information. The messaging format is ASCII. All the fields are pipe (“|”) separated.

Header and Body will consist of delimited String which looks like the following:

♣63=FIX3.0|64=256|65=73|66=2003-01-01155434|1=11|70=10004|4=1000|97=2|19=



Message Header

Tag	Field Name	Reqd	Data Type	Comment
	MessageStartValue	Y	Byte	This field will be start of the message. This field doesn't contain any tag no. Default value = 5 Length = 1 This field is not compressed and has to be prefix at start of compressed FT message.
63	Version	Y	String	To be hardcoded as FIX3.0
64	Message Code	Y	short	Message Code as per message type as mentioned in summary
65	Message Length	Y	short	Length of the whole message excluding MessageStartValue character
66	Message Time	O	string	Date and Time of the message in following format, time is in 24-hour format

Login Request

Tag	Field Name	Reqd	Data Type	Comment
	MessageHeader	Y		Message header to be filled with message code = 101
400	User Type	Y	Integer	Value of 12 for Mobile App Value of 11 for Web
67	User Id	Y	String	
401	Auth Type	Y	Integer	1= Password 2 = Broadcast access token
68	Password/ Broadcast access token	Y	String	Password/ Broadcast access token

Sample Message

63=FIX3.0|64=101|65=66|66=2022-05-04 133022|400=12|67=USER12|68=|

Logon Response

Tag	Field Name	Reqd	Data Type	Comment
	MessageHeader	Y		Message header to be filled with message code = 102
70	Logon Status	Y	Short	Value to identify the logon status. 10000 – Login Successful
97	Days to Expire	O	Short	No. of days for expire. This will be populated within 15 days of expiry of the account. In this case, the logon status will be 10004.
1	Market Segment Id	Y	Short	Login allowed for the user.

Market Data broadcast works on a publish/subscribe model. To receive market data feed, user of the API needs to subscribe for the segment and token for which it needs the market data information

Touchline Request

Tag	Field Name	Reqd	Data Type	Comment
	MessageHeader	Y		Message header to be filled with message code = 206
1	SegmentId	Y	Short	As per Annexure A
7	Security Code/Token	Y	Long	
230	Operation Type	Y	Short	1= Subscribe 2 = Unsubscribe
4	Session Id	Y	String	Session Id obtained at the time of login to application

Requests for multiple segments and token can be sent together

Sample Request

63=FIX3.0|64=206|65=101|66=2022-05-04

134025|1=1\$7=22|1=1\$7=1594|1=3\$7=505537|1=3\$7=500105|1=1\$7=55

3|1=3\$7=526881|1=3\$7=500180|1=1\$7=11630|230=1|4=0x01A40A7258CF071B6365438075D4BB|

Touchline Information

Tag	Field Name	Reqd	Data Type	Comment
	MessageHeader	Y		Message header to be filled with message code = 209
1	SegmentId	Y	Short	As per Annexure A
7	Security Code/Token	Y	Long	
2	Buy Qty	O	Long	
5	Sell Qty	O	Long	
3	Buy Price	O	Long	
6	Sell Price	O	Long	
8	Last Traded Price	O	Long	
9	Last Traded Qty	O	Long	
73	Last Traded Time	O	String	
74	Last Update Time	Y	String	
88	Open Interest	N	Long	
54	NetChangeFromPrevClose	O	String	In Rs
76	Close Price	O	Long	
75	Open Price	O	Long	
77	High Price	O	Long	
78	Low Price	O	Long	
79	Volume	O	Long	
80	Average Traded Price	O	Long	0
81	Total Buy Qty	O	Long	0
82	Total Sell Qty	O	Long	0
93	Life Time High	O	Long	0
94	Life Time Low	O	Long	0
38	Market Type	O	String	Only for NCDEX Spot

BEST FIVE REQUEST

Tag	Field Name	Reqd	Data Type	Comment
	MessageHeader	Y		Message header to be filled with message code = 127
1	SegmentId	Y	Short	As per Annexure A
7	Security Code/Token	Y	Long	
230	Operation Type	Y	Short	1= Subscribe 2 = Unsubscribe
4	Session Id	Y	String	Session Id obtained at the time of login to application

Sample Request

63=FIX3.0|64=127|65=97|66=2023-03-06

185747|1=3|7=500112|230=1|4=0x01A40A7258CF071B6365438075D4BB

BEST FIVE INFORMATION

Tag	Field Name	Reqd	Data Type	Comment
	MessageHeader	Y		Message header to be filled with message code = 128
1	SegmentId	Y	Short	As per Annexure A
7	Security Code/Token	Y	Long	
74	Last Update Time	Y	String	
73	Last Traded Time	O	String	
8	Last Traded Price	O	Long	
9	Last Traded Qty	O	Long	
81	Total Buy Qty	O	Long	
82	Total Sell Qty	O	Long	
76	Close Price	O	Long	
75	Open Price	O	Long	
77	High Price	O	Long	
78	Low Price	O	Long	
79	Volume	O	Long	
80	Average Traded Price	O	Long	
54	NetChangeFromPrevClose	O	String	In Rs
88	Open Interest	N	Long	
380	Daily Price Range	O	String	
93	Life Time High	O	String	
94	Life Time Low	O	String	
This sub structure will be repeated 5 times				
11	B/S Flag	Y	Short	1 for Buy 2 for Sell
14	Price	Y	Long	0
12	Qty	Y	Long	0
37	No Of Orders	Y	Long	0

Sample Code

Sample Code for Socket Connection

```
function ConnectSocket(mode,ip,port){
    try
    {
        objSocket = null;
        objSocket = new WebSocket(mode + '://' + ip + ':' + port);
        objSocket.binaryType = 'arraybuffer';
        objSocket.onopen = socketOnOpen;
        objSocket.onclose = socketOnClose;
        objSocket.onerror = socketOnError;
        objSocket.onmessage = socketOnMessage;

    }
    catch(ex)
    {

    }
}
```

Sample Code for Sending Message

```
function SendMessageOnSocket(msg){
    try
    {
        if(objSocket.readyState == 1)
        {
            objSocket.send(fragmentData(msg));
        }
    }
    catch(ex)
    {

    }
}
```

Sample Code for fragmentData function


```

function fragmentData(_requestPacket) {
    try {
        var _strHead = String.fromCharCode('5'); //5 compression char

        var i;
        var _data = new ArrayBuffer(_strHead.length);
        var _headerBytes = new Uint8Array(_data);

        for (i = 0; i < _strHead.length; i += 1) {
            _headerBytes[i] = _strHead.charCodeAt(i);
        }
        var _baRequest;
        _baRequest = HandleCompressedData(_requestPacket);

        var _length = _baRequest.length + 4;
        var _lenLength = _length.toString().length;
        var _lengthString = "";

        for (i = 0; i < (5 - _lenLength); i++) {
            _lengthString += "0";
        }

        _lengthString += _length.toString();

        _data = new ArrayBuffer(_lengthString.length);
        var _lenBytes = new Uint8Array(_data);

        for (i = 0; i < _lengthString.length; i += 1) {
            _lenBytes[i] = _lengthString.charCodeAt(i);
        }

        var _baActualSend = new Uint8Array(5 + _length);
        _baActualSend.set(_lenBytes);
        _baActualSend.set(_baRequest, 5);

        var _outputStream = new Uint8Array(_headerBytes.length + _baActualSend.length);
        _outputStream.set(_headerBytes);
        _outputStream.set(_baActualSend, 1);
        return _outputStream.buffer;
    }
    catch (e)
    {

    }
};

```

Sample Code to handle compression

```
function  HandleCompressedData(_rawData) {  
  
    try {  
  
        var _data = new ArrayBuffer(_rawData.length);  
  
        var _uint8buf = new Uint8Array(_data);  
  
        for (var i = 0; i < _rawData.length; i += 1) {  
  
            _uint8buf[i] = _rawData.charCodeAt(i) & 0xFF;  
  
        }  
  
        var _compData = Zlib.compress(new Uint8Array(_data), 6);  
  
        return _compData;  
  
    }  
  
    catch (e) {  
  
        }  
  
};
```

Sample Code to send Login Request on Socket Open

```

socketOnOpen = function(){

    var sUsedId = 'USER12';

    var sLoginRequest="63=FIX3.0|64=101|65=66|66=2023-03-06 113929|67=USER12|68=|400=11";

    SendMessageOnSocket((sLoginRequest));

};

```

Sample code for Sending Touchline Request

```

var sMsg="",sMsgToSend="";
sMsg = 1_22,1_2885;
var sTokenArr = sMsg.split(',')
sMsgToSend = "63=FIX3.0|64=206|65=107|66=2023-02-11
19:02:31|4=0x01A40A7258CF071B6365438075D4BB|";

for(var i=0;i < sTokenArr.length;i++)
{
    sMsgToSend += "1=" + (sTokenArr[i].split('_')[0]) + "$7=" + sTokenArr[i].split('_')[1] + "|";
}
sMsgToSend += "230=1";

//Sample String Generated: = 63=FIX3.0|64=206|65=107|66=2023-02-11
19:02:31|1=1$7=22|1=1$7=2885|230=1|4=0x01A40A7258CF071B6365438075D4BB|

SendMessageOnSocket(sMsgToSend);

```

Sample Code to Send Best Five Request

```

var sMsg="",sMsgToSend="";
sMsg = 1_22;

sMsgToSend = "63=FIX3.0|64=127|65=84|66=2023-03-06 19:02:31|1=" + (sMsg.split('_')[0]) + "|7=" +
sMsg.split('_')[1] + "|230=1|4=0x01A40A7258CF071B6365438075D4BB";

//Sample String Generated: = 63=FIX3.0|64=127|65=84|66=2023-03-06
19:02:31|1=1|7=22|230=1|4=0x01A40A7258CF071B6365438075D4BB

SendMessageOnSocket(sMsgToSend);

```

Sample Response Processing

```

function socketOnMessage(_event){
try
{
    var baProcessData = null;
    var intRawPktLen;
    var intCompLen = 0;
    var _response, isBroken = false;
    var totalPacketLength = 0;

    if (_event.data instanceof ArrayBuffer) {

        var dataReceived = null;
        var dataPacketLengthList = [];
        if (baOldData == null)
            dataReceived = new Uint8Array(_event.data);
        else {
            dataReceived = AppendOrCopyBuffer(baOldData, _event.data);
            baOldData = null;
        }

        intRawPktLen = dataReceived.byteLength;
        var i = 0;
        if (intRawPktLen > 5) {
            while (i < intRawPktLen) {
                if (dataReceived[i] == 5 || dataReceived[i] == 2) {
                    var strPacketLength = String.fromCharCode.apply(null, dataReceived.subarray(i + 1, i + 6));
                    if (strPacketLength.length == 5) {
                        var packetLength = parseInt(strPacketLength, 10);
                        dataPacketLengthList.push(packetLength + 6);
                        totalPacketLength += packetLength + 6;
                        i = i + 6 + packetLength;
                    }
                    else {
                        baOldData = dataReceived.subarray(i, intRawPktLen);
                        isBroken = true;
                        break;
                    }
                }
            }
        }
        else
            baOldData = dataReceived;

        if (intRawPktLen == totalPacketLength) {
            // split and pass to zlib to uncompress
            for (var i = 0, j = 0, k = dataPacketLengthList[0], len = dataPacketLengthList.length; i < len ; i++) {

                var uncompData = dataReceived.subarray(j, k);
                ProcessSocketMessage(uncompData);

                j = k;
                k = k + dataPacketLengthList[i + 1];
            }
            baOldData = null;
        }
    }
}

```

```

else {
    var i = 0, j = 0;
    var k = (dataPacketLengthList.length > 0 ? dataPacketLengthList[0] : 0);

    if (!isBroken) {
        for (var len = dataPacketLengthList.length; i < len - 1; i++) {
            var uncompData = dataReceived.subarray(j, k);
            ProcessSocketMessage(uncompData);
            j = k;
            k = k + dataPacketLengthList[i + 1];
        }

        if (i == dataPacketLengthList.length - 1) {
            baOldData = null;
            baOldData = dataReceived.subarray(j, k);
            if (baOldData[0] != 5 && baOldData[0] != 2)
                alert();
        }
    }
    else {
        for (var len = dataPacketLengthList.length; i < len; i++) {
            var uncompData = dataReceived.subarray(j, k);
            ProcessSocketMessage(uncompData);

            j = k;
            k = k + dataPacketLengthList[i + 1];
        }
        isBroken = false;
    }
}
}
}
}
}
catch(ex)
{
}
}

function ProcessSocketMessage(uncompData) {
    _response = DeCompressData(uncompData);
    if (_response == undefined) {
        console.log(_response);
    }
    //remove End of response char
    var intTmtrIndex = _response.indexOf("\u0000");
    if (intTmtrIndex != -1) {
        _response = _response.substr(0, intTmtrIndex);
    }
    //split multi response packet with Start of response char C_S_CHAR2
    var arrData = _response.split("\u0002");
    var intDataCount = arrData.length;
    for (var intDataCntr = 0; intDataCntr < intDataCount; intDataCntr++) {
        if (arrData[intDataCntr] != "") {
            console.log("Packet: " + arrData[intDataCntr]);
        }
    }
}
}
}

```

Annexure A

Exchange	SegmentId
NSE-CASH	1
NSE-DERIVATIVES	2
BSE-CASH	3
MCX-DERIVATIVES (COMMODITIES)	5
MCX-SPOT (COMMODITIES)	6
NCDEX-DERIVATIVES(COMMODITIES)	7
NCDEX-SPOT (COMMODITIES)	8
NSECDS-DERIVATIVES(CURRENCIES)	13
NSE-CDS(SPOT)	14